Guía de estudio y exposición - Proyecto 2a

Base de datos asíncrona con Python y MongoDB

Adolfo Escobar Pereyra - 362839 - 8CC2 Cómputo Paralelo y Distribuido Profesor: José Saúl de Lira Miramontes

Esta guía sirve para: entender el proyecto, correrlo en terminal y exponerlo. Si solo lees 3 secciones, lee la 1, la 5 y la 6.

1. De qué trata (tu resumen de 30 segundos)

> "Hice dos aplicaciones que administran películas en MongoDB con las > MISMAS funciones: una síncrona con PyMongo clásico y una asíncrona con > PyMongo Async. La asíncrona usa TaskGroup, gather, Semaphore y > bulk_write para hacer muchas operaciones a la vez. Luego las comparé con > un benchmark: como el trabajo es esperar la red (I/O-bound), la versión > asíncrona gana porque traslapa las esperas en lugar de sumarlas."

Palabras clave que el profesor quiere oír: **AsyncMongoClient**, **asyncio.gather**, **asyncio.Task Group**, **asyncio.Semaphore**, **bulk_write**, **I/O-bound**, **sample_mflix**.

2. Mapa del proyecto (qué hace cada archivo)

Archivo	Qué es	Lo importante adentro
`config.py`	Ajustes compartidos	URI, las 2 colecciones, `get_sync_client`, `get_async_client`, `indices`
`dataset/generador_peliculas.py`	Crea datos de prueba estilo mflix	`generar_peliculas(n, semilla)`, `en_lotes`
`sync_app/app_sync.py`	App SÍNCRONA (`MongoClient`)	`SyncMoviesDB`: carga, CRUD, consultas, `demo`, `menu`
`async_app/app_async.py`	App ASÍNCRONA (`AsyncMongoClient`)	`AsyncMoviesDB`: lo mismo pero concurrente
`comparacion/benchmark.py`	Compara ambas (requiere MongoDB)	Experimentos A, B, C + tabla con speedup
`comparacion/demo_concurrencia.py`	Demo simulada SIN MongoDB	Prueba el concepto con `sleep`
`scripts/verificar_conexion.py`	Diagnóstico	Prueba conexión y cuenta docs
`scripts/cargar_sample_mflix.py`	Importador opcional	Carga un `movies.json` descargado
`tests/test_sin_bd.py`	14 pruebas sin servidor	Fakes en memoria, verifican todo
`reporte/*.pdf`	Reporte del proyecto	Lo que se entrega + este código

Cómo se conectan:

```
config.py + generador  <-  los usan todos
     |
     +-- sync_app/app_sync.py ---+---> comparacion/benchmark.py
     +-- async_app/app_async.py -+
```

3. Cómo funciona cada parte (para explicar con confianza)

3.1 `config.py` - las dos colecciones (idea clave de diseño)

- **Lectura:** `sample_mflix.movies` - el dataset oficial, SOLO se lee.
- **Escritura:** `proyecto2a.movies_carga` - aquí van carga masiva, CRUD y benchmarks. Se puede borrar sin miedo.
- `elegir_coleccion_lectura()`: "si el sample existe y tiene docs, úsalo; si no, usa la colección de trabajo". Así todo funciona **con o sin** el sample oficial instalado.
- `get_sync_client()` devuelve `MongoClient`; `get_async_client()` devuelve `AsyncMongoClient` (requiere `pymongo >= 4.10`).
- `ensure_*_indexes()`: crea índices en `title`, `year`, `genres`, `imdb.rating` para que filtros y ordenamientos sean rápidos.

3.2 `generador_peliculas.py` - datos deterministas

- `generar_pelicula(rng)`: un documento con el esquema de mflix (`title`, `year`, `genres`, `directors`, `cast`, `imdb.rating`...).
- `generar_peliculas(n, semilla=362839)`: misma semilla = mismos datos siempre. Esto hace los benchmarks **repetibles**.
- `en_lotes(docs, 1000)`: parte 5000 docs en 5 bloques de 1000 para enviarlos por `bulk_write`.

3.3 `sync_app/app_sync.py` - la app síncrona

Clase `SyncMoviesDB`, todo secuencial (una operación tras otra):

- **Carga masiva:** `carga_masiva_bulk_write` - por cada lote hace `bulk_write([InsertOne(doc), ...])` y espera la respuesta.
- **CRUD:** `crear_pelicula`, `obtener_por_id`, `obtener_por_titulo`, `listar` (paginado con `skip+limit`), `actualizar_pelicula`, `actualizar_muchos`, `eliminar_pelicula`, `eliminar_muchos`.
- **Consultas:** `consulta_por_genero`, `consulta_por_rango_anios`, `consulta_top_rating` (ordena por `imdb.rating`), `consulta_buscar_titulo` (regex), `consulta_agregacion_por_genero` (`$unwind+$group+$sort`), `consulta_promedio_por_anio`, `consulta_top_directores`.
- `demo()` corre carga + CRUD + consultas sola; `menu()` es interactivo.

3.4 `async_app/app_async.py` - la app asíncrona (el corazón del proyecto)

Clase `AsyncMoviesDB` con la **misma API** (mismos métodos). Diferencias:

- **Carga masiva con TaskGroup** (`carga_masiva_taskgroup`): crea UNA TAREA POR LOTE dentro de `async with asyncio.TaskGroup()`. Cada tarea ejecuta `_insertar_lote` = `bulk_write([InsertOne...])` bajo el `Semaphore`. Al salir del bloque, todas terminaron.

- **Variante con gather** (`carga_masiva_gather`): lo mismo pero con `await asyncio.gather(*tareas)`.

- **Consultas concurrentes** (`consultas_concurrentes_gather`): mezcla 5 tipos de consulta y las lanza todas juntas; devuelve resultados + tiempo. Hay versión gemela con `TaskGroup`.

- **Bulk concurrente:** `bulk_actualizar_ratings` (`UpdateOne`) y `bulk_eliminar` (`DeleteOne`) dividen en lotes y los procesan con `gather` + `Semaphore`.

- **Detalle fino (di esto si preguntan):** el `Semaphore` se crea DENTRO de cada operación, no en el constructor, porque un `Semaphore` queda ligado al *event loop* en uso y el objeto se reutiliza desde varios loops (benchmark y pruebas).

- Lecturas async: `find()` devuelve cursor async -> `await cursor.to_list(n)`. Igual para `aggregate()`.

3.5 `benchmark.py` - la comparación justa

- Corre la **misma carga** en ambas apps y se queda con el **mejor de 3** tiempos por experimento:

- **A. Carga masiva:** N docs en lotes (secuencial vs TaskGroup).

- **B. Consultas:** Q consultas mezcladas (una tras otra vs gather).

- **C. Update masivo:** N `UpdateOne` (secuencial vs concurrente).

- Cada experimento limpia la colección antes, para partir del mismo estado.

- Todo lo asíncrono corre en **un solo `asyncio.run`** (un event loop).

- Imprime tabla con `speedup = tiempo_sync / tiempo_async` y guarda `comparacion/resultados_benchmark.json`.

- Si no hay MongoDB, no se cae: explica las 3 opciones (local / Atlas / demo simulada).

3.6 `demo_concurrencia.py` y `tests/` - sin MongoDB

- **Demo simulada:** cada "consulta" es un `sleep` de ~50 ms (simula la latencia de red). Mide sync vs gather vs TaskGroup. Resultado real medido: 1.66 s -> 0.19 s

(speedup x8.9 con concurrencia 10).

- **Tests:** 14 pruebas con colecciones falsas en memoria que imitan a PyMongo. Verifican CRUD, consultas, agregaciones, carga masiva, concurrencia (el Semaphore sí limita: máx <= 4 con tope 4) y que el benchmark usa la misma carga en ambas ramas.

4. Las 4 primitivas exigidas (qué decir de cada una)

Primitiva	Qué es (en una frase)	Dónde está
`asyncio.gather()`	Lanza varias corrutinas juntas y espera todas	`carga_masiva_gather`, `consultas_concurrentes_gather`, bulk update/delete
`asyncio.TaskGroup()`	Bloque que crea tareas hijas y las espera de forma estructurada	`carga_masiva_taskgroup`, `consultas_concurrentes_taskgroup`
`asyncio.Semaphore(n)`	Semáforo: máximo n operaciones a la vez, no satura al servidor	Todas las operaciones concurrentes
`bulk_write()`	Muchas escrituras en una sola petición al servidor	Carga (`InsertOne`), updates (`UpdateOne`), deletes (`DeleteOne`)

Frase para exponer: *"gather y TaskGroup generan el paralelismo, el Semaphore lo limita para no saturar a MongoDB, y bulk_write hace eficiente cada envío por lotes."*

5. Comandos (todo lo que necesitas en terminal)

5.1 Instalación (una sola vez)

```
cd "/Users/adolfo/Desktop/Computo Paralelo y Distribuido/Proyecto2a_MongoDB_362839"
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
cp .env.example .env
```

5.2 MongoDB local (una sola vez, elige una opción)

```
# Opción A: Homebrew (recomendada en Mac)
brew tap mongodb/brew
brew trust mongodb/brew
brew install mongodb-community
brew services start mongodb-community

# Opción B: Docker (si lo tienes)
docker run -d -p 27017:27017 --name mongo mongo

# Opción C: Atlas (nube gratis) -> crea clúster, saca tu URI y ponla en .env:
# MONGO_URI=mongodb+srv://usuario:password@cluster0.xxxx.mongodb.net/
```

Verificar conexión y ver conteo de colecciones:

```
source .venv/bin/activate
python -m scripts.verificar_conexion
```

(Opcional, sample oficial en Atlas: en tu clúster -> botón "Load Sample Dataset". Si no lo cargas, no pasa nada: el generador incluido llena la colección de trabajo con el mismo esquema.)

5.3 Corridas principales (para exponer y para medir)

```
source .venv/bin/activate

# Demo síncrona: carga + CRUD + consultas
python -m sync_app.app_sync --demo --n 2000

# Demo asíncrona: carga (TaskGroup) + CRUD + consultas (gather y TaskGroup)
python -m async_app.app_async --demo --n 2000 --concurrencia 10

# BENCHMARK comparativo (el número que va al reporte, tarda ~1-3 min)
python -m comparacion.benchmark --n 5000 --lote 1000 --consultas 50

# Rápido (1 repetición, para probar)
python -m comparacion.benchmark --n 2000 --consultas 20 --rapido
```

5.4 Sin MongoDB (verificación + concepto)

```
source .venv/bin/activate

# 14 pruebas, ~0.2 s
python -m unittest discover -s tests -v

# Demo simulada de concurrencia (speedup ~ igual a la concurrencia)
python -m comparacion.demo_concurrencia --consultas 40 --concurrencia 10
```

5.5 Menús interactivos (para jugar / mostrar CRUD en vivo)

```
python -m sync_app.app_sync
python -m async_app.app_async
```

5.6 Regenerar el PDF del reporte (si editas el .md)

```
pip install fpdf2
python reporte/generar_reporte_pdf.py
```

6. Guion de exposición sugerido (8-10 minutos)

- **(1 min) Qué hice:** lee tu resumen de la sección 1. Muestra el mapa de archivos (sección 2).

- **(1 min) Diseño:** explica las dos colecciones (no toco el sample original) y el generador determinista con semilla 362839.

- **(2 min) App síncrona:** muestra `SyncMoviesDB` y corre `python -m sync_app.app_sync --demo --n 2000`. Señala el tiempo.

- **(3 min) App asíncrona (lo importante):** muestra `carga_masiva_taskgroup` y `consultas_concurrentes_gather`, explica gather/TaskGroup/Semaphore/bulk_write con la frase de la sección 4. Corre `python -m async_app.app_async --demo --n 2000`.

- **(2 min) Comparación:** muestra la tabla del benchmark (experimentos A, B, C + speedups) y concluye: *"gana async porque el trabajo es I/O-bound; el speedup crece con la latencia"*.

- **Cierre:** tests 14/14 + demo simulada como evidencia extra.

Si algo falla en vivo (sin red, Mongo apagado): corre los comandos de la sección 5.4 - funcionan siempre y demuestran el concepto.

7. Preguntas probables del profesor (y respuestas cortas)

¿Por qué la asíncrona es más rápida? Porque cada operación pasa la mayor parte del tiempo esperando la red; async traslapa esas esperas, sync las suma. Trabajo I/O-bound.

¿Y si fuera CPU-bound (cálculos)? Async NO ayudaría por el GIL; habría que usar procesos o hilos. Por eso elegí async: una BD es I/O-bound.

¿Para qué el Semaphore? ¿Qué pasa sin él? Limita a N operaciones simultáneas. Sin él, miles de tareas saturan el pool de conexiones y al servidor, y el rendimiento cae o hay errores.

¿gather o TaskGroup, cuál es mejor? Hacen lo mismo aquí; TaskGroup es más moderno y estructurado (si una tarea falla, cancela el resto y reporta bien los errores). Implementé ambos para comparar: miden prácticamente igual.

¿Por qué bulk_write y no insert_one en ciclo? Un `bulk_write` manda cientos de escrituras en UNA petición: menos viajes de red, mucho más throughput.

¿Por qué dos colecciones? Para no contaminar ni borrar el `sample_mflix` original: solo lo leo; todo lo que escribo va a `proyecto2a.movies_carga`.

¿Cómo garantizas que la comparación es justa? Misma API, mismos datos (generador determinista), mismo estado inicial (limpio antes de cada experimento), misma carga (verificado: 10 consultas -> 90 docs en ambas ramas) y mejor de 3 repeticiones.

¿Qué índices creaste y por qué? `title`, `year`, `genres`, `imdb.rating`, `directors`: son los

campos que filtro y ordeno en las consultas.

¿Qué versión de Python/pymongo? Python 3.11+ (por `TaskGroup`), `pymongo >= 4.10` (por `AsyncMongoClient`). Probado con Python 3.14 y pymongo 4.19.

8. Si algo falla (soluciones rápidas)

Síntoma	Causa probable	Solución
`No se pudo conectar a MongoDB`	Mongo apagado o URI mal	`brew services start mongodb-community`; revisa `.env` / `--uri`
`TaskGroup` no existe	Python < 3.11	Usa `python3.11+`; el código trae fallback a `gather` de todos modos
`No module named 'pymongo'`	Olvidaste el venv	`source .venv/bin/activate` + `pip install -r requirements.txt`
Benchmark lento en Atlas gratis	Latencia + tier compartido	Normal; usa `--rapido` o menos `--n` para probar
`Load Sample Dataset` no aparece	Clúster M0 viejo o sin opción	No lo necesitas: el generador llena la colección de trabajo

Fin de la guía. Archivos fuente: este documento vive en `Proyecto2a_MongoDB_362839/GUIA_EXPOSICION.md` (+ `.pdf` con el mismo nombre).